

3차시 · AI 기획 전략

AI 시대 MVP 기획 전략

빠르게 만들기 전에, 올바른 것을 정한다.

앞에선 '어떻게'(Codex로 만들기) — 지금부터는 '무엇을'(만들 것 정하기).

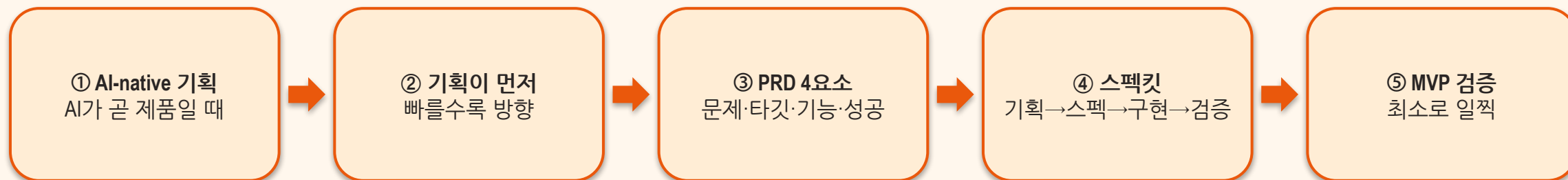
“바이브 코딩은 빠른데 — 방향이 틀리면?”

빠르게 '틀린 것'을 만든다. 그래서 기획이 먼저다

이 차시에서 얻는 것

오늘의 지도 — 5단계 흐름

기획 = 에이전트에게 줄 정확한 방향을 준비하는 일



SECTION 01

AI-native 서비스 기획

AI가 곧 제품일 때 어디를 노릴 것인가

기능으로서의 AI vs 제품으로서의 AI

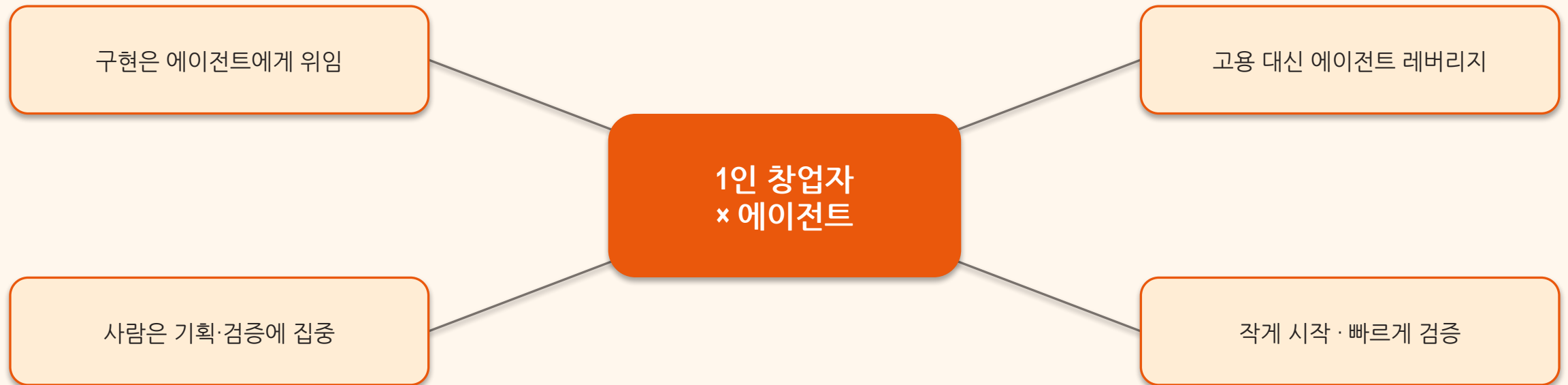
기능으로서의 AI

- 기존 제품에 AI 버튼 하나 추가
- AI가 없어도 제품은 동작
- 차별화가 약하기 쉬움
- '있으면 좋은' 부가기능

제품으로서의 AI

- AI가 핵심 가치를 만든다
- 빼면 제품이 성립 안 됨
- 경험의 중심에 AI가 박힘
- '없으면 안 되는' 본질

1인·소수 팀이 에이전트를 레버리지로



기능 10개(X) → 결정적 경험 1개(O)

사용자가 “와” 하는 단 하나의 순간을 한 문장으로 정의

- 기능 나열의 함정 욕심이 MVP를 망치는 가장 흔한 이유
- 찾는 법: “이걸 보면 사용자가 다시 온다”는 순간
- 예: “사진 한 장 올리면 3초 만에 메뉴 추천”
- 타깃이 좁을수록 WOW가 또렷해진다

AI-native 기획 체크 — 좁게, 또렷하게

- ☐ AI가 핵심 가치를 만드는가?
- ☐ wow를 한 문장으로 말할 수 있는가?
- ☐ 1인이 만들 범위로 좁혔는가?
- → AI가 핵심을 만드는 곳에, wow 하나를 좁게

SECTION 02

왜 기획이 먼저인가

속도의 시대일수록 방향의 가치가 커진다

빠르게 만든다 = 빠르게 맞거나 틀린다

✗ 기획 없이 바로 코딩

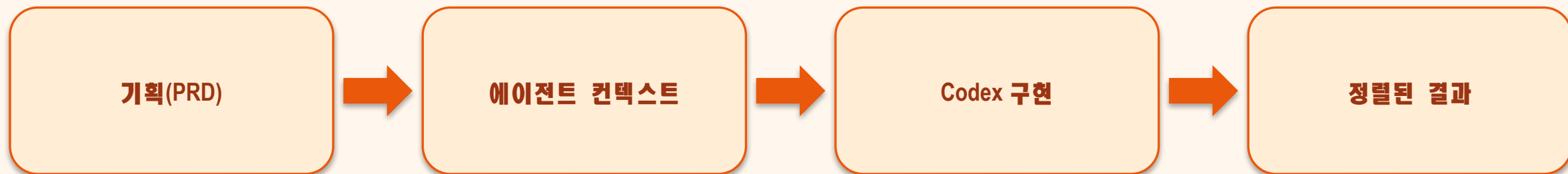
- 범위가 계속 변진다
- 매번 다른 걸 만든다
- 검증 기준이 없다
- 많이 만들수록 되돌리기 비싸다

VS

✓ 기획을 먼저

- 방향을 일찍 정할수록 싸다
- 같은 기준으로 만들고 검증한다
- 분량이 아니라 명확함
- 한 장 PRD로도 충분하다

PRD가 곧 에이전트의 컨텍스트



기획은 에이전트에게 줄 **정확한 방향**을 준비하는 일. 'What을 분명히 = 좋은 위임'. 기획이 좋으면 결과가 좋다.

SECTION 03

PRD — 무엇을 · 누구를 · 왜

PRD(Product Requirements Document) = 제품 요구사항 문서

무엇을 / 누구를 위해 / 왜 만드는지 정리한 한 장

- 팀의 합의 + 에이전트의 컨텍스트, 두 역할을 동시에
- 사람과 AI가 같은 문서를 본다
- 처음부터 완벽할 필요 없음 검증하며 고치는 살아있는 문서
- 오늘 이 PRD를 4차시에서 Codex에 그대로 입력 → 초안 생성

What, not How — Cagan의 제1원칙

PRD는 '무엇을' 설명한다 '어떻게'는 설명하지 않는다

- 문제·행동·경험을 정의하고, 구현 방법은 에이전트/엔지니어에게 위임
- 20년 된 업계 표준 원칙(Marty Cagan, SVPG) — '좋은 위임'과 같은 원리
- How까지 지정하면 에이전트의 더 나은 해법을 막는다

PRD 4요소



① 어떤 불편을 푸는가 · ② 누가 쓰는가 · ③ 최소 기능 1~3개 · ④ 무엇이 '성공'인가(측정 가능) + 가드레일: 비목표(안 만드는 것)

모호함을 구체로 — 각 요소의 ✖ / ✔

✖ 나쁜 PRD

- 문제: “사람들이 불편하다”
- 타겟: “모두”
- 기능: 욕심껏 10개 나열
- 성공: “좋은 반응”

✔ 좋은 PRD

- 문제: “점심 메뉴 고르기 귀찮다”
- 타겟: “혼밥 직장인”
- 기능: 버튼을 누르면 점심 메뉴를 대신 골라준다.
- 성공: “5명 중 3명이 다음날 다시 쓴다”

한 단계 위의 PRD — 유저 스토리 · 비목표 · 수용 기준

각 요소를 프로답게 업그레이드

- 유저 스토리: "[타겟]으로서 [행동]하고 싶다, [이유] 때문에"
- 비목표: 안 만드는 것 명시(회원가입❌ 지도❌ 결제❌) — 범위 번짐 방지 (ChatPRD)
- 수용 기준: "~하면 통과" 체크 문장 "클릭→1초 안에 표시" (Cagan 릴리스 기준)

위에서 아래로 — 쓰기 전 '숙제' 먼저, 그리고 다섯 칸

1

문제 정의

어떤 불편/필요를 푸는가 한 문장 (쓰기 전 숙제: 고객·경쟁·기술)

2

타겟 사용자

누가 쓰는가 유저 스토리 1줄('~로서 ~하고 싶다, ~ 때문에')

3

핵심 기능 + 비목표

최소 기능 1~3개 + wow 명시 · 안 만드는 것(비목표)도 함께 적는다

4

성공 기준

'~하면 통과' 수용 기준으로 측정 가능하게(성능·사용성·신뢰성)

Cagan의 PRD — 4영역과 10단계

How To Write a Good PRD(2005) — 오늘 만들어보는 PRD는 이 구조의 경량판

- 4영역: 제품 목적 · 기능 · 릴리스 기준 · 일정 ↔ 문제·타겟·기능·비목표·성공
- 10단계: 숙제 → 목적 → 사용자 → 원칙 → 프로토타입 검증 → 가정 의심 → 쓰기 → 우선순위 → 완결성 → 살아있는 문서
- 쓰기 전 숙제(고객·경쟁·기술) · 제품 원칙 1줄("항상 3초 안에 답한다")

이 양식 그대로 4차시에서 사용

다섯 줄만 채우면 방향이 고정된다

[제품명] PRD
- 문제:
- 타겟: (유저 스토리 1줄)
- 핵심 기능(1~3): (wow 명시)
- 비목표(안 만드는 것):
- 성공 기준: (측정 가능한 수용 기준)

- 4차시 배포 자료와 동일 양식 한쪽이 바뀌면 양쪽 같이 유지
- 분량보다 명확함이 목표 한 장이면 충분

채워진 예 — 점심 랜덤 추천기

점심 랜덤 추천기 PRD

- 문제: 혼밥 직장인이 매일 메뉴 고르기에 피로
- 타겟: 20~30대 혼밥 직장인 - "3초 안에 메뉴를 정하고 싶다"
- 핵심 기능: ① 랜덤 추천 (wow: 3초 결정) ② 음식 종류 필터
- 비목표: 회원가입 ✕ · 지도/✕ □ □□ ✕ □
- 성공 기준: 클릭→1초 안에 표시 / 데모 5명 중 3명 "다시 쓰겠다"

- 1차시 실습과 연결되는 친숙한 예 작고 구체적
- 성공 기준이 숫자로 측정 가능하다는 점에 주목

Amazon PR/FAQ — 만들기 전에 보도자료부터

Working Backwards — 고객 경험에서 거꾸로

- 출시일 '보도자료 + FAQ'를 먼저 쓴다 Kindle·AWS·Prime Video가 이렇게 시작
- 예: "○○사, 점심 고민을 3초로 줄이는 '점심 랜덤 추천기' 출시"
- wow를 한 문장으로 못 쓰면 보도자료도 못 쓴다 같은 원리의 대기업 버전 (심화 옵션)

에이전트가 읽기 좋은 PRD — 3원칙

LLM은 입력의 구조와 디테일에 의존한다 (ChatPRD)

- ① 구조가 곧 품질 제목·불릿·표로, 긴 산문 금지
- ② 모호어 사냥 "빠르게"→"3초 안에" · "편하게"→"클릭 1번" · "예쁘게"→"모바일 1열 카드"
- ③ 커지면 6섹션으로 개요 / 타겟·유저스토리 / 기능 / 기술 제약 / 비기능 / 비목표

PRD 작성 체크

- ☐ 문제를 한 문장으로 말할 수 있는가?
- ☐ 타깃이 유저 스토리로 적혔는가? ('모두' 금지)
- ☐ 핵심 기능이 하나의 flow로 좁혀졌는가? (wow 명시)
- ☐ 비목표(안 만드는 것)를 명시했는가?
- ☐ 성공 기준이 측정 가능한 수용 기준인가?
- ☐ 모호어('빠르게·편하게')가 남아있지 않은가?

SECTION 04

스펙킷 워크플로

명세에서 구현으로 기획과 코드 사이의 다리

기획 → 스펙 → 구현 → 검증, 그리고 반복



PRD(무엇을) → 스펙(무엇과 어떻게를 연결) → 구현 → 검증. 스펙(명세)을 단일 진실로 삼아 사람도 에이전트도 같은 기준을 본다.

명세 중심으로 기획 → 설계 → 구현을 잇기

GitHub **Spec Kit** — Spec-Driven Development 툴킷

- **specify** CLI + 슬래시 명령으로 워크플로를 구조화
- 흐름: Specify → Plan → Tasks → Implement (각 단계가 Markdown 산출물)
- 각 산출물이 다음 단계의 구조화된 컨텍스트가 된다
- Codex·Claude 등 여러 에이전트 지원 도구는 갈아끼울 수 있다

실제 화면

github.com/github/spec-kit — 스펙킷 저장소

uvx --from git+https://github.com/github/spec-kit.git specify init ... 으로 설치 없이 바로 실행

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search or jump to...

Sign in

Sign up

github / spec-kit

Public

Notifications

Fork 10.5k

Star 119k

<> Code

Issues 228

Pull requests 176

Discussions

Actions

Projects

Models

Security and quality

Insights

main

21 Branches

183 Tags

Go to file

<> Code

WOLIKIMCHENG and root

feat(scripts): add Python check-prerequisites ...

73f77c2 · 15 hours ago

1,318 Commits

.devcontainer	chore: retire Roo Code integration — extension shut down ...	last week
.github	chore(deps): bump actions/setup-dotnet from 5.3.0 to 5.4....	yesterday
.specify/memory	feat: add specify bundle command (#3070)	3 weeks ago
docs	docs: drop stale kimi KIMI.md->AGENTS.md migration not...	5 days ago
examples/bundles	feat: add specify bundle command (#3070)	3 weeks ago
extensions	Support namespaced git feature branch templates (#3293)	17 hours ago
integrations	fix(integrations): add zed to discovery catalog.json (#3266)	last week
media	fix: broken media files	9 months ago
newsletters	Add June 2026 newsletter (#3289)	last week
presets	chore: retire Roo Code integration — extension shut down ...	last week
scripts	feat(scripts): add Python check-prerequisites PoC (#3302)	15 hours ago
src/specify_cli	fix(integrations): hermes honors SPECKIT_INTEGRATION_...	15 hours ago
templates	feat: make agent-context extension a full opt-in (#3097)	last week
tests	feat(scripts): add Python check-prerequisites PoC (#3302)	15 hours ago

About

Toolkit to help you get started with Spec-Driven Development

github.github.com/spec-kit/

engineering development ai spec copilot prd spec-driven

Readme

MIT license

Code of conduct

Contributing

Security policy

Cite this repository

Activity

Custom properties

119k stars

602 watching

10.5k forks

Report repository

Releases 181

Spec Kit - 0.12.5 Latest

16 hours ago

+ 180 releases

스펙킷 4단계 — 작성·검토·위임·검증

1

스펙 작성

PRD를 바탕으로 화면·동작·데이터를 명세로

2

검토

팀이 함께 빠진 동작·경계 조건 점검 여기서 잡으면 싸다

3

구현 위임

스펙을 Codex에 주고 구현 컨텍스트가 풍부해 결과가 정렬

4

검증·반복

결과를 스펙 기준으로 검증, 어긋나면 스펙/구현 수정

핵심은 '명세 → 구현 → 검증' 흐름

정식 스펙킷 — 캠프 기본 경로

- Spec Kit + Codex — 캠프 실습 전체가 이 경로
- 3차시 실습: \$speckit-* 8개 명령 전부 체험
- 4차시: constitution → specify → clarify → checklist
- 5차시: plan → tasks → implement

경량 대안 — 도구가 부담될 때

- PRD + 잘 쓴 프롬프트(2차시 5요소)
- AGENTS.md에 규칙, 프롬프트에 스펙 핵심
- 5분 실험·설치가 어려운 환경용 흐름은 동일

SECTION 05

MVP Validation

만들고 끝이 아니다 최소로 만들어 일찍 검증한다

가설을 검증할 최소 기능 제품

“최소”의 핵심은 기능 수가 아니라 wow 1개

- MVP = 가설을 검증할 최소 기능 제품
- 가설 사고: “사용자는 X를 원할 것이다”를 검증 대상으로 명시
- 무엇이 참이면 성공인가를 한 문장으로 (PRD '성공 기준'과 연결)
- wow 1개만 동작하게 결과지 제거

프로토타입이 곧 스펙

"스펙의 대부분은 고화질 프로토타입이어야 한다" — Cagan, 2005

- 20년 전엔 프로토타입이 비쌌다 에이전트가 몇 분에 만드는 지금, 예언이 기본값
- 가정을 적고, 의심하라(Cagan Step 6) — 적지 않은 가정은 검증 못 한다
- 만들어서 검증한다

가설 → MVP → 데모 → 학습, 그리고 반복



사람에게 보여주고 배운다. 적게 만들고 일찍 배운다 = 위험·비용 최소화. 살아있는 PRD와 한 흐름.

빠른 검증의 가치 — 흔한 함정 피하기

- 적게 투자 → 큰 학습 = 위험·비용 최소화
- **×** 검증 없이 기능만 키우기
- **×** 행복 경로만 보기
- → 최소로 만들어 일찍 검증, 배워서 고친다

빠르게 만들기 전에, 올바른 것을 한 장으로

AI-native 기획 · 기획 우선 · PRD 4요소+비목표 · 스펙킷 · MVP 검증

- 한 문장 정의 연습: “우리는 [타겟]의 [문제]를 [wow]로 푼다”
- □ 문제·타겟·wow를 한 문장으로
- □ PRD 핵심 4항목 채우기

4·5차시 — 오늘 배운 흐름을 그대로

- **4차시:** 팀별 MVP 워크샵 MVP Canvas + Codex로 PRD 초안 생성
- **5차시:** PRD를 스펙 → 구현으로 (오늘의 스펙킷 흐름)
- 팀 구성·주제 선정: AI가 핵심 가치인 작은 것
- 산출물: 한 문장 정의 + PRD 초안 → 다음 차시 입력

“방향을 정한 사람만이, 에이전트를 제대로 부린다.”

기획 → 스펙 → 구현 → 배포로 이어지는 길